



2025/2026

NOM : VONGSANGA

PRÉNOM : Clément

☐ Conduite de projet (S6)

☒ Avant projet d'ingénieur (S8)

☐ Projet d'ingénieur (S10)

Rapport de projet

ENTREPRISE : FEV

SUJET DU PROJET : Conception et déploiement d'un framework logiciel centralisé, sécurisé et automatisé pour l'unification des outils d'ingénierie

MAÎTRE D'APPRENTISSAGE : Smail BRIK

TUTEUR PÉDAGOGIQUE : Jérémie HAQUET



CADRE RÉSERVÉ POUR LES STAGES DE FIN D'ÉTUDES (STAGE INGÉNIEUR)

OPTION

☐ EOC ☒ ISI ☐ LEC

CONFIDENTIALITÉ

mon rapport est confidentiel niveau ☒ 1 (moy) ☐ 2 (élevé)

Un mail a été envoyé par votre tuteur à Mme Baronnet (Dijon), Mme Delevaux (Paris) ou Mme Omont (Angers) au moins un avant votre soutenance, sinon l'ESEO retient le niveau 0 cf. fiche d'encadrement.

DOMAINE

ENTREPRISE

☒ Cybersécurité, data science et intelligence artificielle

☐ Véhicules intelligents

☐ Santé

☐ Développement durable et villes intelligente

☐ Industrie du futur

☐ Ingénierie d'affaires

Engagement De Non Plagiat

Je soussigné, Clément Vongsanga, étudiant(e) à l'ESEO, atteste avoir pris connaissance du contenu du Règlement intérieur de l'École et de l'engagement de « non-plagiat ». Je déclare m'y conformer dans le cadre de la rédaction de ce document. Je déclare sur l'honneur que le contenu du présent mémoire est original et reflète mon travail personnel. J'atteste que les citations sont correctement signalées par des guillemets et que les sources de tous les emprunts ponctuels à d'autres auteur(e)s, textuels ou non textuels, sont indiquées. Le non-respect de cet engagement m'exposerait à des sanctions dont j'ai bien pris connaissance.

Fait à Voisins le Bretonneux le 22/05/2026

Signature de l'auteur du rapport



Déclaration d'utilisation de l'Intelligence Artificielle

Dans le cadre de la rédaction de ce rapport de stage, j'ai utilisé des outils d'intelligence artificielle, pour les tâches suivantes :

- Génération de la structure du rapport, correction grammaticale, génération de texte

L'utilisation de ces outils a été réalisée sous la supervision de mon maître d'apprentissage et a fait l'objet d'une concertation pour garantir l'intégrité du travail. Les parties du rapport concernées par l'utilisation de l'IA sont les suivantes :

- Sommaire, Rapport principal

Après avoir utilisé ces outils, j'ai personnellement révisé et édité le contenu généré pour m'assurer de sa pertinence et de son exactitude. Je prends l'entière responsabilité du contenu final de ce rapport.

Fait à Voisins le Bretonneux le 22/05/2026

Signature de l'auteur du rapport



Remerciements

Je tiens à remercier chaleureusement toutes les personnes qui m'ont accompagné et soutenu tout au long de cette période d'alternance.

Je remercie en particulier mon maître d'apprentissage, Smail BRIK, pour sa disponibilité, ses conseils avisés et la confiance qu'il m'a accordée tout au long de ma mission. Son accompagnement a été essentiel à ma progression professionnelle et personnelle.

Je remercie également l'ensemble de l'équipe de FEV, pour leur accueil chaleureux, leur bienveillance et leur esprit de collaboration qui m'ont permis de m'intégrer rapidement et de m'impliquer pleinement dans mes projets.

Je souhaite aussi exprimer ma gratitude envers l'ESEO et son équipe pédagogique pour le suivi attentif tout au long de mon cursus, ainsi que pour la qualité de la formation dispensée.

Sommaire

I. FICHE SYNTHÈSE.....	5
II. ABSTRACT.....	6
III. INTRODUCTION.....	7
IV. CHAPITRE 1 : CONTEXTE PROFESSIONNEL ET PROBLÉMATIQUE.....	9
a. L'environnement FEV et l'équipe Outils.....	9
1. Le cadre industriel : FEV France.....	9
2. Intégration et rôle au sein de l'équipe Outils.....	9
b. État de l'art : Un écosystème fragmenté et non sécurisé.....	10
1. Fragmentation logicielle et ergonomie hétérogène.....	10
2. Risques de cybersécurité et obsolescence technique.....	11
3. Analyse du risque lié à la dette technique et aux exécutable.....	11
4. Limites opérationnelles de la chaîne de test HIL.....	11
5. Scénario d'usage actuel : La préparation d'une campagne de test HIL.....	12
c. Objectifs SMART du Projet Ingénieur (PING).....	12
V. CHAPITRE 2 : TRAVAUX PRÉPARATOIRES ET FAISABILITÉ TECHNIQUE.....	14
a. Mise en conformité Cybersécurité : Traitement de la dette technique.....	14
b. Interopérabilité des données : Le convertisseur A2L vers YAML.....	14
c. Détail de l'algorithme d'extraction hybride.....	15
d. Innovation d'Interface : Architecture Hybride HTML / MATLAB.....	15
1. Le moteur de rendu dynamique (Frontend) :.....	16
2. La communication bidirectionnelle (Backend MATLAB) :.....	16
VI. CHAPITRE 3 : CADRAGE ET ARCHITECTURE DU FRAMEWORK UNIFIÉ (PING)....	19
a. Architecture globale de la Toolbox : Un Hub local connecté.....	19
b. Sécurisation des données : Stratégie Zero Trust et Entra ID.....	19
c. Automatisation de la chaîne HIL : Flux de bout en bout.....	21
VII. CHAPITRE 4 : INGÉNIERIE LOGICIELLE, QUALITÉ ET PLANIFICATION.....	23
a. Gestion de configuration et travail collaboratif.....	23
b. Usine logicielle : Stratégie CI/CD et Assurance Qualité.....	24
c. Planification et Roadmap (S9 - S10).....	26
VIII. CHAPITRE 5 : VALIDATION DES COMPÉTENCES INGÉNIEUR ESEO.....	27
a. Analyser et Diagnostiquer.....	27
b. Concevoir.....	27
c. Produire et Réaliser.....	27
d. Valider.....	27
e. Piloter (et s'organiser).....	28
f. S'adapter.....	28
g. Communiquer.....	28
IX. CONCLUSION.....	29
X. RÉFÉRENCES BIBLIOGRAPHIQUES ET WEBOGRAPHIE.....	30
XI. GLOSSAIRE.....	31

I. FICHE SYNTHÈSE

Sujet: Conception et déploiement d'un framework logiciel centralisé, sécurisé et automatisé pour l'unification des outils d'ingénierie.

Entreprise: FEV France, Trappes, France.

Dates du projet: Février 2026 - Mai 2026.

Principales personnes avec qui l'apprenti a travaillé: Smail BRIK (Maître d'apprentissage, développeur sénior) et Jérémie HAQUET (Tuteur pédagogique ESEO).

Résumé :

- **Problématique** : L'équipe « Outils » de FEV France gère un écosystème de plus de 20 applications développées en MATLAB ou Python 3. Bien que globalement moderne, cet écosystème souffrait d'une fragmentation opérationnelle et d'une dette technique critique localisée sur les outils de la RP App, restés sous Python 2.7.2. Cette obsolescence ciblée engendrait des failles de sécurité, des blocages d'exécutables par l'IT et imposait des saisies manuelles fastidieuses lors de la préparation des tests Hardware-in-the-Loop (HIL).
- **Objectifs SMART** : Concevoir et déployer un framework unique, moderne et sécurisé, la FEVio Toolbox, pour fédérer ces 20 outils et éradiquer les processus manuels dans le flux de configuration HIL. Le projet vise 100 % de conformité avec la politique cyber de l'entreprise d'ici août 2027.
- **Démarche et Réalisations** : Durant ce semestre, j'ai résorbé la dette technique en migrant l'ensemble des scripts de la RP App vers Python 3.14.3. J'ai développé un parseur hybride (Regex/ASAM) convertissant les données de calibration A2L en YAML interopérable. Pour l'interface globale, j'ai conçu une architecture hybride (Front HTML/JS fluide, Back MATLAB via le composant uihtml et des flux JSON). Enfin, j'ai défini l'architecture d'un modèle d'accès sécurisé Zero-Trust via le protocole OAuth 2.0 PKCE (Microsoft Entra ID), dont l'implémentation est prévue au prochain semestre pour le téléchargement des modèles Simulink sur SharePoint.
- **Résultats quantitatifs** : Les Preuves de Concept (PoC) ont validé la stabilité de la communication bidirectionnelle. L'automatisation des flux a permis de réduire le temps de configuration de la RP App de deux semaines à une seule demi-journée, représentant un gain de productivité supérieur à 90 %.
- **Conclusion et implications** : Ce semestre S8 valide la viabilité technique du framework. Ce socle robuste ouvre la voie aux phases de déploiement et d'intégration de la totalité des outils dans la Toolbox lors des semestres S9 et S10, qui s'accompagneront de l'implémentation effective d'Entra ID et de la mise en place d'une usine logicielle CI/CD (GitHub Actions).

II. ABSTRACT

Title: Design and Deployment of a Centralized, Secure, and Automated Software Framework for Engineering Tool Unification

Company: FEV France

Supervisors: Smail BRIK (Company Mentor), Jérémie HAQUET (Academic Tutor)

- **Motivation:** In automotive testing, Hardware-in-the-Loop (HIL) simulation requires efficient and error-free configuration workflows. At FEV France, the "Tools" team manages an ecosystem of over twenty utilities built on MATLAB or Python 3. However, software fragmentation and critical technical debt were isolated within the legacy RP App utilities, which still relied on deprecated Python 2.7.2. This specific bottleneck caused cybersecurity vulnerabilities and local executable blocks by corporate IT policies, slowing down development agility and HIL test bench preparation.
- **Key Question:** How can a scattered ecosystem of engineering tools be unified into a secure, modern desktop application while resolving the legacy RP App Python debt and automating HIL configuration?
- **Objectives:** The project aims to develop a unified framework named FEVio Toolbox. Semestre 8 focused on migrating the RP App communication scripts to Python 3.14.3, validating a dynamic and license-free HTML/MATLAB user interface, and designing a secure data layer architecture to access Simulink models.
- **Approach:** The framework uses a decoupled architecture. A web frontend (HTML/CSS/JS) is dynamically generated via JSON files and communicates bi-directionally with a MATLAB backend using the native uihtml component (DataChanged events). Simulink model metadata (.slx) is extracted using native MATLAB APIs, and Python processes the data into an Excel pivot file (PIV). For future security, the architecture defines a Zero-Trust model via Microsoft Entra ID (OAuth 2.0 PKCE) that will be implemented to securely download models from SharePoint.
- **Results:** The Semestre 8 Proof of Concept successfully cleared the targeted technical debt by stabilizing the RP App stack under Python 3.14.3. The hybrid A2L-to-YAML parser was validated. Operationally, the automated pipeline reduced the test bench preparation time for the RP App from two weeks down to less than a single business day (a >90% time reduction).
- **Conclusions:** All core technology blocks are validated. This reliable foundation allows for the full-scale migration of the remaining tools into the FEVio Toolbox during Semestres 9 and 10, focusing on the active implementation of the Entra ID security layer, continuous pipeline testing (CI/CD via GitHub Actions), and automated reporting tools.

III. INTRODUCTION

Dans un secteur automobile en pleine mutation, marqué par l'électrification des chaînes de traction et la complexification des logiciels embarqués, la fiabilité et la rapidité des campagnes de validation logicielle constituent des enjeux industriels majeurs. Pour répondre à ces défis, les ingénieurs d'essais s'appuient au quotidien sur des outils d'automatisation métier afin d'accélérer les processus de test, notamment sur les bancs de simulation Hardware-in-the-Loop (HIL).

C'est dans ce cadre que s'inscrit ce rapport de Semestre 8, qui pose les bases d'un Projet Ingénieur (PING) d'une durée de 18 mois au sein du département eMobility de FEV France, sur le site de Trappes. La mission principale de l'équipe « Outils » consiste à développer et maintenir des applications pour assister les collaborateurs dans la standardisation et l'automatisation de leurs tâches répétitives. Cependant, l'écosystème logiciel actuel souffre d'une forte fragmentation : une vingtaine d'outils cohabitent sans charte graphique unifiée, manquent parfois de documentation et reposent sur des technologies subissant une obsolescence technique critique au regard des nouvelles normes de cybersécurité de l'entreprise.

L'objectif de ce Projet Ingénieur est de concevoir et de déployer un framework unifié, baptisé FEVio Toolbox. Ce projet ne se limite pas à une simple refonte d'interfaces graphiques ; il vise à standardiser l'ensemble des outils métiers sous une architecture hybride moderne (HTML/MATLAB/Python) et à sécuriser de manière native (Cybersecurity by Design) les flux d'accès aux bases de données confidentielles via un modèle Zero Trust interconnecté à l'infrastructure Microsoft Entra ID de l'entreprise.

Ce document de cadrage a pour but de valider la faisabilité technique et la dimension d'ingénierie de cette démarche. Pour ce faire, le rapport est structuré en cinq chapitres principaux :

- **Le Chapitre 1** présente le contexte professionnel de FEV France, l'organisation de l'équipe et détaillera la problématique industrielle liée à la fragmentation des outils.
- **Le Chapitre 2** dresse le bilan des travaux préparatoires menés au cours du Semestre 8, faisant office de Preuves de Concept (PoC) techniques indispensables au projet (migration de scripts d'automatisation, création d'un convertisseur de données et développement d'un moteur de rendu d'interface dynamique).
- **Le Chapitre 3** détaille l'architecture cible du Framework Unifié, en ciblant spécifiquement la sécurisation des flux de données et l'automatisation de la chaîne de test HIL.
- **Le Chapitre 4** aborde la méthodologie d'ingénierie logicielle retenue, la mise en place d'une usine logicielle via un pipeline d'intégration et de déploiement

continus (CI/CD), ainsi que la planification macroscopique des jalons pour les semestres 9 et 10.

- Enfin, **le Chapitre 5** met en perspective ce projet avec le référentiel des sept compétences d'ingénieur exigées par l'ESEO, démontrant l'adéquation de la mission avec les exigences du diplôme.

IV. CHAPITRE 1 : CONTEXTE PROFESSIONNEL ET PROBLÉMATIQUE

a. L'environnement FEV et l'équipe Outils

1. Le cadre industriel : FEV France

Le groupe FEV est un acteur international majeur de l'ingénierie automobile et des nouvelles mobilités. Fondé en 1978, il compte aujourd'hui environ 7 000 collaborateurs répartis dans plus de 40 pays. FEV intervient principalement en tant que prestataire de Recherche et Développement (R&D) avancée pour le compte de grands constructeurs et équipementiers mondiaux, couvrant la conception de chaînes de traction (thermiques, hybrides, électriques, hydrogène), le développement de logiciels embarqués et la fourniture de moyens d'essais.

En France, la filiale FEV France s'appuie sur environ 400 collaborateurs. Le site de Trappes (78), où se déroule mon apprentissage, constitue un pôle stratégique majeur. Il concentre des activités de pointe liées à l'eMobility, notamment le développement de systèmes de gestion de batteries (BMS), la validation d'onduleurs, et l'exploitation de bancs de test physiques et virtuels pour les chaînes de traction électriques.

2. Intégration et rôle au sein de l'équipe Outils

Au sein du département eMobility et rattaché au projet transversal XP_legacy, j'évolue en tant qu'Apprenti Développeur Outils. L'équipe « Outils » est une structure restreinte composée de trois personnes : mon maître d'apprentissage (développeur sénior), un second ingénieur logiciel, et moi-même. Bien que notre cœur de mission soit le développement logiciel de support, les autres membres de l'équipe interviennent en parallèle sur d'autres disciplines d'ingénierie au sein du département, notamment sur de la conception système. Cette polyvalence permet à l'équipe de maintenir un ancrage fort avec les réalités et les besoins concrets du métier de l'électromobilité.

La mission de notre entité est de concevoir, déployer et maintenir des applications logicielles dites « supports ». Ces outils sont mis à disposition des ingénieurs d'essais et de validation en France, mais également dans des filiales internationales comme FEV Italie ou FEV Maroc. L'objectif de notre équipe est double :

- L'optimisation de l'efficacité opérationnelle : par l'automatisation des tâches répétitives ou chronophages, permettant aux ingénieurs de se concentrer sur l'analyse technique.
- La standardisation : par la mise en place de formats de données, de structures de fichiers et de rapports unifiés, garantissant la cohérence et la qualité des livrables de FEV face aux exigences des clients.

L'équipe fonctionne de manière pragmatique sans méthodologie Agile formalisée. L'activité est rythmée par des réunions hebdomadaires de synchronisation permettant de dresser l'état d'avancement des développements, de prioriser les demandes d'évolutions et d'organiser l'entraide technique. Les signalements de dysfonctionnements (bugs) et les demandes de nouveaux outils sont actuellement consignés par les utilisateurs dans un fichier partagé Excel. Ce mode de gestion traditionnel montre aujourd'hui ses limites, non seulement techniques, mais aussi humaines : l'outil étant rigide et déconnecté du quotidien des ingénieurs, il est fréquemment oublié et n'est pas rempli de manière rigoureuse.

b. État de l'art : Un écosystème fragmenté et non sécurisé

Une analyse approfondie de l'existant met en évidence une dette technique et organisationnelle importante, structurée autour de trois axes principaux.

1. Fragmentation logicielle et ergonomie hétérogène

Le patrimoine logiciel du projet XP_legacy comprend environ une vingtaine d'outils métiers développés de manière itérative au fil des ans pour répondre à des besoins sectoriels immédiats. Cette approche organique a conduit à une forte fragmentation :

- **Hétérogénéité des structures de code** : Les outils cohabitent sans architecture logicielle commune. Certains sont écrits exclusivement en Python, d'autres reposent sur des scripts MATLAB ou des macros VBA.
- **Dispersion fonctionnelle** : Parmi cette vingtaine d'applications, on retrouve des briques essentielles au quotidien des ingénieurs, telles que l'outil de comparaison de modèles Simulink ou le script de conversion de fichiers de calibration A2L combinés à des fichiers de données hexadécimales (.hex) vers des fichiers exécutables MATLAB (.m). Malgré leur utilité critique, ces outils restent isolés les uns des autres.
- **Disparité des Interfaces Utilisateurs (UI)** : L'expérience utilisateur n'est pas standardisée. Certains outils disposent d'interfaces développées sous MATLAB App Designer (visuellement datées et restrictives en termes de licences), d'autres sous des interfaces graphiques Python rudimentaires, tandis qu'une part importante d'entre eux s'exécute uniquement en ligne de commande (CLI).
- **Le défi de la conduite du changement** : Au-delà de l'aspect purement technique, cette fragmentation engendre un problème majeur de communication et d'appropriation. L'absence d'une plateforme unique et de guides utilisateurs standardisés fait que les collaborateurs ignorent parfois l'existence même de certains outils. Ce manque de visibilité historique a freiné l'intérêt des équipes pour ces solutions logicielles, renforçant le besoin de proposer une interface moderne et dynamique pour susciter à nouveau l'engagement des utilisateurs.

2. Risques de cybersécurité et obsolescence technique

Le deuxième constat majeur concerne la conformité aux exigences de sécurité de l'information (IT) du groupe FEV :

- **Obsolescence des runtimes** : Plusieurs applications critiques, à l'instar de la suite automatisée pour le logiciel Intecrio (RP App), reposent historiquement sur Python 2.7.2. Cette version n'étant plus supportée, elle expose l'infrastructure à des vulnérabilités de sécurité et ne répond plus aux critères de conformité de l'entreprise.
- **Exposition des données sources** : La distribution actuelle des fichiers de configuration ou des modèles d'ingénierie s'effectue sans contrôle d'accès unifié. Les fichiers sources sont stockés sur des répertoires partagés classiques, ce qui induit un risque élevé d'altération accidentelle (écrasement de fichiers) ou de fuite de propriété intellectuelle.
- **Blocage des exécutables locaux** : Pour simplifier le déploiement, les scripts Python sont compilés en exécutables autonomes (.exe) via l'utilitaire PyInstaller. En l'absence de signature numérique par un certificat approuvé par le service IT de FEV, ces outils sont régulièrement détectés comme des menaces par les agents de sécurité des postes de travail (antivirus/EDR) et purement bloqués, paralysant l'activité des ingénieurs.

3. Analyse du risque lié à la dette technique et aux exécutables

L'obsolescence des environnements d'exécution ne se résume pas à un simple problème de performance, elle constitue une faille de sécurité majeure. Le maintien d'applications basées sur Python 2.7.2 expose le système d'information de l'entreprise à des vulnérabilités connues (CVE - Common Vulnerabilities and Exposures) qui ne bénéficient plus de correctifs officiels depuis le 1er janvier 2020. Dans le cadre de projets liés à l'ingénierie automobile, où la confidentialité des algorithmes de contrôle moteur ou de gestion de batterie (BMS) est critique, cette exposition est inacceptable pour l'IT.

De plus, la politique de sécurité des terminaux (Endpoint Security) chez FEV s'est considérablement durcie. Les solutions de type EDR (Endpoint Detection and Response) analysent le comportement des exécutables locaux. Les outils actuels, compilés via des utilitaires open-source sans signature cryptographique d'entreprise, sont parfois mis en quarantaine. Cela oblige les ingénieurs d'essais à formuler des demandes de dérogation chronophages auprès du support informatique pour chaque nouvelle mise à jour d'un outil, paralysant ainsi l'agilité des équipes de développement et de validation.

4. Limites opérationnelles de la chaîne de test HIL

Le processus de configuration des bancs de test Hardware-in-the-Loop (HIL) illustre parfaitement les limites de l'automatisation actuelle. Pour préparer une campagne

d'essais, les ingénieurs doivent extraire manuellement des données issues de modèles Simulink (.slx) et de fichiers de description de calculateurs (A2L) pour remplir un fichier Excel pivot (appelé fichier PIV).

Un outil en version Bêta est actuellement utilisé pour transcrire ce fichier PIV en scripts Python de test. Cependant, l'absence de documentation rigoureuse sur la structure de ces fichiers d'entrée rend le générateur instable : les scripts produits souffrent de défauts de fiabilité (erreurs de nommage des variables génériques issues de l'A2L), obligeant les équipes à effectuer des corrections manuelles ultérieures dans le code généré. Ce flux de travail hybride (saisie manuelle → génération instable → correction manuelle) représente une perte de temps conséquente et une source majeure d'introduction d'erreurs humaines dans les boucles de validation.

5. Scénario d'usage actuel : La préparation d'une campagne de test HIL

Pour bien saisir l'impact de ces limites opérationnelles, il convient d'analyser le flux de travail actuel d'un ingénieur préparant une campagne d'essais sur un banc HIL. Ce processus se décompose en plusieurs étapes manuelles hautement faillibles :

- Recherche des sources : L'ingénieur doit d'abord localiser la bonne version du modèle Simulink (.slx) et le fichier de calibration associé (.a2l) sur les serveurs partagés de l'entreprise. En l'absence de gestionnaire de version centralisé pour ces fichiers, le risque d'utiliser une version obsolète est élevé.
- Extraction manuelle : L'utilisateur ouvre le modèle Simulink. Il doit naviguer visuellement dans l'arborescence complexe des blocs (qui peut comporter plusieurs dizaines de sous-systèmes) pour identifier les ports d'entrée/sortie et les paramètres de calibration pertinents pour son test.
- Renseignement du fichier PIV : Ces informations (noms des blocs, types de données, valeurs par défaut) sont ensuite recopiées manuellement, ou via des copier-coller successifs, dans le fichier Excel pivot (PIV). Cette tâche, en plus d'être chronophage, est la principale source d'erreurs typographiques.
- Génération et correction : L'outil Bêta actuel ingère ce fichier PIV pour générer un script Python. Cependant, si le fichier A2L contient des alias ou si l'ingénieur a fait une erreur de casse dans l'Excel, le script généré fera appel à des variables inexistantes sur le banc HIL, provoquant le crash du test au moment de son exécution.

Ce fonctionnement en "silos" justifie à lui seul la nécessité de développer une chaîne d'outils interconnectée et automatisée de bout en bout.

c. Objectifs SMART du Projet Ingénieur (PING)

Pour résoudre cette problématique, le projet PING fixe des objectifs clairs, mesurables et planifiés :

- **S (Spécifique)** : Concevoir une Toolbox globale (interface hybride HTML/MATLAB) centralisant la vingtaine d'outils existants dans un modèle ergonomique unique, intégrant un module de récupération de données sécurisé, une chaîne d'automatisation HIL fiable et un système de mise à jour transparente.
- **M (Mesurable)** : Atteindre un taux de migration de 100 % des outils critiques vers la nouvelle interface, éliminer totalement les saisies manuelles lors de la phase Simulink → PIV → Python, et garantir un taux d'exécution sans blocage de sécurité des fichiers .exe.
- **A (Atteignable)** : S'appuyer sur les preuves de concept déjà validées au S8 (migration Python 3.14.3 réussie pour Intecrio, validité du moteur de rendu JSON/JS, convertisseur A2L fonctionnel).
- **R (Réaliste)** : Répondre directement aux directives de standardisation de FEV et aux exigences de conformité cyber de la direction informatique.
- **T (Temporellement défini)** : Un lotissement sur 18 mois, s'étendant du Semestre 8 (analyse et PoC) jusqu'à la clôture en août 2027 (passation et soutenance).

V. CHAPITRE 2 : TRAVAUX PRÉPARATOIRES ET FAISABILITÉ TECHNIQUE

Afin de sécuriser le déploiement du futur framework unifié (PING) et d'en valider la viabilité technique, le Semestre 8 a été consacré à la réalisation de plusieurs travaux préparatoires. Ces missions ont fait office de Preuves de Concept (PoC) permettant de lever les risques technologiques identifiés lors de l'état de l'art, tout en assurant le Maintien en Conditions Opérationnelles (MCO) des outils actuels.

a. Mise en conformité Cybersécurité : Traitement de la dette technique

La première étape critique de ce semestre concerne la mise en conformité de l'existant vis-à-vis des nouvelles directives de cybersécurité du groupe FEV.

Le socle d'automatisation du logiciel Intecrio (intégré à la RP App) reposait historiquement sur des scripts développés en Python 2.7.2. La fin de vie officielle de cette version exposait le système à des vulnérabilités non patchées. J'ai donc piloté la migration complète de ce code source vers la version Python 3.14.3.

La difficulté technique majeure de cette migration a résidé dans l'adaptation de la bibliothèque propriétaire logicielle fournie par société ETAS, indispensable pour la communication entre Python et Intecrio. Cette dépendance nécessitait une conversion rigoureuse pour assurer sa stabilité sous Python 3. En revanche, ayant anticipé cette transition technologique, j'avais veillé à développer mes propres scripts récents en respectant au maximum la syntaxe et les standards de la version 3 dès leur conception, ce qui a considérablement réduit la dette technique lors de la migration. Cette mise à niveau valide la robustesse de l'automatisation Intecrio, brique essentielle des futures chaînes de tests.

b. Interopérabilité des données : Le convertisseur A2L vers YAML

Pour atteindre l'objectif du PING visant le "zéro saisie manuelle" dans la configuration des bancs HIL, il était impératif de prouver notre capacité à extraire et manipuler les données issues des modèles d'ingénierie bruts.

Les ingénieurs s'appuient sur des fichiers A2L, un standard de l'industrie automobile décrivant les paramètres des calculateurs électroniques (ECU). Or, ces fichiers complexes sont inexploitablement en l'état par nos générateurs de tests. J'ai donc développé un module de conversion automatisé.

Techniquement, la lecture et l'extraction de ces données reposent sur une approche hybride. Le script Python combine l'utilisation d'une bibliothèque spécialisée dans le parsing de la norme ASAM/A2L avec l'usage d'expressions régulières (Regex) écrites sur mesure. Cela permet de traiter les cas spécifiques non couverts par le standard.

L'outil génère in fine un fichier de configuration structuré au format YAML. Ce fichier agit comme un pont d'interopérabilité : il crée une équivalence formelle entre le nom spécifique de la variable dans le fichier PIV (le tableau Excel des ingénieurs) et son nom générique correspondant dans l'A2L. Cette traduction est la clé de voûte de notre future chaîne d'automatisation.

c. Détail de l'algorithme d'extraction hybride

Le format A2L (ASAM MCD-2 MC) est un fichier de description standardisé principalement utilisé dans les environnements de développement et d'étalonnage. Un fichier de production peut peser plusieurs dizaines de mégaoctets et contenir des centaines de milliers de lignes décrivant la topologie mémoire d'un calculateur (ECU).

Pour garantir des performances d'exécution optimales sans saturer la mémoire (RAM) lors du traitement, j'ai implémenté une lecture séquentielle optimisée. L'algorithme cible spécifiquement les blocs MEASUREMENT (pour les variables lues sur le banc) et CHARACTERISTIC (pour les paramètres calibrables).

L'extraction combine une bibliothèque de parsing ASAM et des expressions régulières (Regex) écrites sur mesure pour capturer les métadonnées spécifiques à l'environnement FEV. Voici un extrait simplifié de la logique d'extraction développée en Python :

```
import re
import yaml

# Compilation de la Regex pour optimiser les performances de recherche
# Capture le nom de la variable et son adresse hexadécimale
regex_measurement = re.compile(r'/begin\s+MEASUREMENT\s+([A-Za-z0-9_]+)
[\s\S]*?ECU_ADDRESS\s+(0x[0-9A-Fa-f]+)')

def extract_a2l_metadata(filepath):
    extracted_data = {}
    # Lecture sécurisée et gestion de l'encodage spécifique aux fichiers
    # A2L
    with open(filepath, 'r', encoding='utf-8', errors='ignore') as file:
        content = file.read()
        matches = regex_measurement.findall(content)

        for var_name, hex_address in matches:
            extracted_data[var_name] = {
                'address': hex_address,
                'type': 'MEASUREMENT'
            }
    return extracted_data
```

d. Innovation d'Interface : Architecture Hybride HTML / MATLAB

Le défi technique le plus complexe du semestre a consisté à concevoir l'architecture de la future interface graphique de la Toolbox. L'utilisation des outils natifs (comme MATLAB App Designer) a été rapidement écartée en raison d'un rendu visuel

vieillissant et de coûts de licences inadaptés à un déploiement massif. Pour contourner ces limitations, j'ai conçu une architecture hybride, séparant strictement la logique d'affichage (Frontend) de la logique métier (Backend).

1. Le moteur de rendu dynamique (Frontend) :

J'ai abandonné le principe des interfaces codées "en dur". Le fonctionnement repose sur la lecture d'un fichier de configuration JSON définissant les éléments souhaités. Un script JavaScript analyse le fichier JSON et génère dynamiquement l'interface HTML correspondante sur une grille fluide à 12 colonnes.

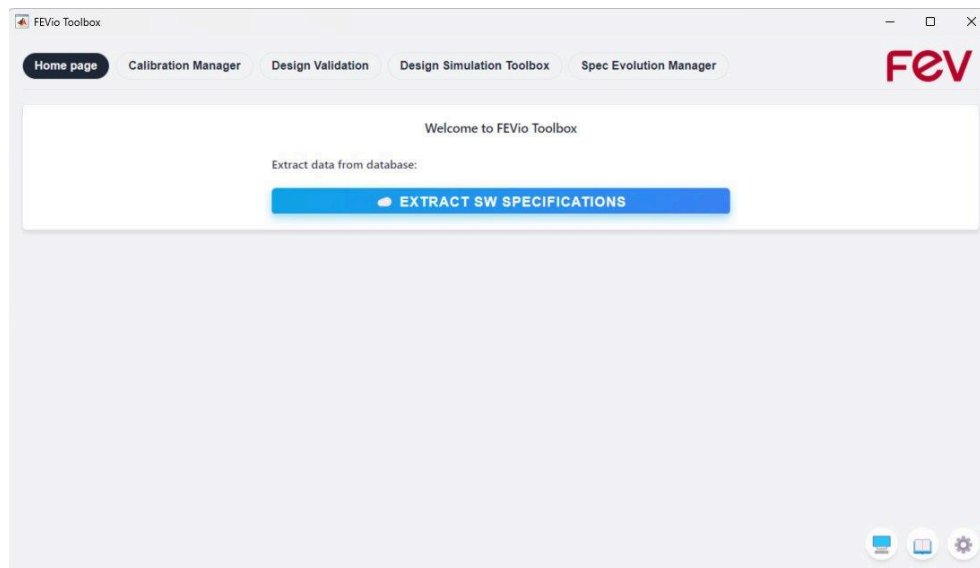


Figure 1 : Exemple d'interface générée

Voici un exemple simplifié illustrant la structure du JSON permettant de déclarer dynamiquement un bouton dans l'interface :

```
{  
  "type": "button",  
  "id": "btn_run_conversion",  
  "label": "Run Conversion"  
}
```

2. La communication bidirectionnelle (Backend MATLAB) :

Du côté de MATLAB, l'affichage et l'intégration du web s'appuient sur le composant graphique uihhtml, instancié au sein d'une figure moderne (uifigure). Ce composant charge la page index.html et sert de passerelle de communication bidirectionnelle.

Concrètement, le code MATLAB utilise la propriété .Data du composant uihhtml pour transmettre le fichier JSON au moteur JavaScript. En retour, MATLAB écoute en permanence l'événement DataChanged lié au composant. Lorsqu'un utilisateur

interagit avec l'interface (par exemple en cliquant sur le bouton défini ci-dessus), le JavaScript modifie ces données, ce qui déclenche l'événement côté MATLAB. Le backend intercepte alors l'ID de l'action (btn_run_conversion) et exécute automatiquement la fonction associée.

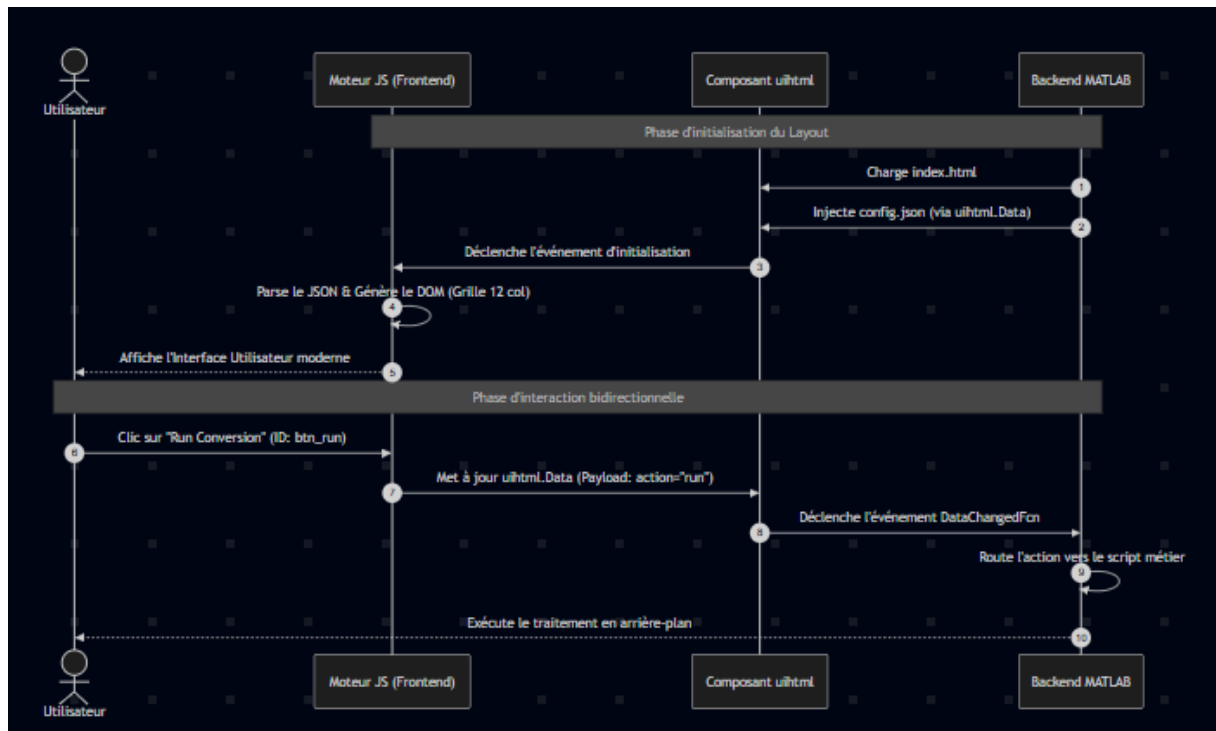
Cette architecture modulaire valide de manière définitive la faisabilité de la FEVio Toolbox, offrant une flexibilité totale tout en conservant la puissance de calcul de MATLAB en arrière-plan.

e. Séquence de communication bidirectionnelle et gestion d'état

Le défi principal de cette architecture hybride résidait dans la synchronisation de l'état entre le navigateur web (DOM) et l'espace de travail (Workspace) MATLAB. La communication n'étant pas nativement asynchrone pour ce type de processus, elle nécessite une gestion stricte des événements (Event Listeners).

Le flux de données se déroule selon un cycle de vie précis :

- Initialisation (Backend vers Frontend) : MATLAB lit le fichier de configuration JSON et l'injecte dans la propriété .Data du composant uihtml.
- Rendu Dynamique : Le moteur JavaScript détecte ce changement, analyse le JSON et génère dynamiquement les balises HTML dans le navigateur web DOM selon le système de grille fluide.
- Interaction Utilisateur : Lorsqu'un clic est effectué (par exemple sur le bouton de conversion), le JavaScript intercepte l'événement onClick.
- Callback (Frontend vers Backend) : Le script JS modifie à nouveau l'objet de données partagées en y insérant une charge utile (payload), par exemple : {"action": "run_conversion", "id": "btn_1"}.
- Exécution du routage : MATLAB détecte cette modification via son callback DataChangedFcn, isole l'identifiant de l'action, et lance l'algorithme métier correspondant.



VI. CHAPITRE 3 : CADRAGE ET ARCHITECTURE DU FRAMEWORK UNIFIÉ (PING)

Les travaux préparatoires ayant validé la viabilité des différentes briques technologiques, ce chapitre détaille l'architecture logicielle cible du Projet Ingénieur. La solution, baptisée FEVio Toolbox, se présente comme un écosystème hybride conçu pour répondre aux problématiques de fragmentation, de sécurité et d'automatisation des processus HIL.

a. Architecture globale de la Toolbox : Un Hub local connecté

La FEVio Toolbox ne sera pas conçue comme une application Web hébergée sur un serveur distant, mais comme un client lourd local (Desktop Application) connecté au Cloud de l'entreprise. Ce choix d'architecture permet de conserver la puissance de calcul locale de la machine de l'ingénieur, indispensable pour l'exécution rapide des scripts MATLAB et Python.

Stratégie de déploiement :

Dans un premier temps, afin de garantir une agilité maximale et de contourner les blocages de sécurité liés aux exécutables non signés (.exe), la Toolbox sera distribuée sous la forme d'une archive compressée (ZIP) contenant l'arborescence des scripts et l'interface. Les utilisateurs lanceront l'environnement via un script d'amorçage. À terme, en collaboration avec le service IT, l'objectif est d'encapsuler cette architecture dans un exécutable autonome et certifié.

Fonctionnement hybride :

L'architecture de la Toolbox repose sur un triptyque technologique :

- Le Frontend (HTML/CSS/JS) : Gère le rendu de l'interface dynamique via le moteur JSON détaillé précédemment.
- Le routeur principal (MATLAB) : Héberge l'interface web via uihtml et dispatche les événements utilisateurs.
- Le moteur de traitement (Python & MATLAB) : Exécute les algorithmes métiers complexes en arrière-plan.

b. Sécurisation des données : Stratégie Zero Trust et Entra ID

L'une des innovations majeures de ce projet réside dans la création d'un module de récupération sécurisé des modèles de conception. Historiquement stockés sur des répertoires réseaux classiques, ces modèles confidentiels seront désormais centralisés sur un site SharePoint sécurisé appartenant à l'infrastructure Microsoft 365 de FEV.

Pour y accéder depuis la Toolbox locale sans compromettre la sécurité, j'ai défini une architecture basée sur le modèle Zero Trust (Confiance Zéro) :

- Délégation d'authentification : L'outil ne stocke aucun mot de passe ou clé API en dur (Modèle Public Client). Lorsqu'un ingénieur souhaite télécharger un modèle, la Toolbox délègue l'authentification à Microsoft Entra ID (anciennement Azure AD). L'utilisateur s'authentifie via le portail officiel de l'entreprise (SSO/MFA).
- Principe du moindre privilège : Une fois authentifiée, l'application reçoit un jeton d'accès (Access Token) éphémère. Ce jeton est strictement limité au scope Files.Read.All sur le SharePoint cible.
- Intégrité garantie : Cette limitation technique rend impossible toute altération, suppression ou écrasement des modèles sources sur le cloud depuis la Toolbox, assurant une protection totale du patrimoine de données de l'entreprise. L'ingénieur travaille exclusivement sur une copie rapatriée en local.

c. Mécanisme d'authentification OAuth 2.0 avec extension PKCE

L'intégration de la Toolbox avec Microsoft Entra ID pour l'accès sécurisé au SharePoint de l'entreprise ne peut pas reposer sur un flux OAuth 2.0 standard pour "applications confidentielles" (qui exige un identifiant secret enregistré côté serveur). La FEVio Toolbox étant un client local (Public Client) distribué sur les postes des ingénieurs, stocker un "Client Secret" dans les fichiers locaux ou le code source représenterait une faille majeure de sécurité par rétro-ingénierie.

Pour sécuriser ce canal, j'ai sélectionné le protocole OAuth 2.0 avec l'extension PKCE (Proof of Key for Code Exchange, norme RFC 7636). Ce protocole garantit l'intégrité de l'échange de jetons sans stockage de secret statique.

Le cycle de sécurisation des requêtes suit les étapes suivantes :

- Génération des secrets dynamiques : À chaque demande de connexion, la Toolbox génère localement une clé cryptographique aléatoire à usage unique appelée code_verifier, ainsi que son empreinte hachée en SHA-256 appelée code_challenge.
- Requête d'autorisation : La Toolbox ouvre le navigateur système de l'utilisateur vers le point de terminaison d'autorisation de Microsoft Entra ID, en transmettant l'identifiant de l'application (Client ID) et le code_challenge.
- Authentification d'entreprise : L'ingénieur s'authentifie sur l'interface sécurisée de FEV (via Authentification Unique / SSO et validation multifacteur MFA). Une fois validé, Entra ID renvoie un code d'autorisation temporaire à la Toolbox via un port d'écoute local (localhost).
- Échange sécurisé : La Toolbox renvoie ce code d'autorisation accompagné du code_verifier original non haché. Entra ID valide la correspondance cryptographique.
- Émission du jeton : Le serveur émet un jeton d'accès sous format JWT (JSON Web Token). Ce jeton, signé numériquement par FEV et à durée de vie éphémère (60 minutes), contient les droits stricts Files.Read.All. La Toolbox

l'injecte ensuite dans l'en-tête HTTP Authorization: Bearer <token> pour interroger l'API Microsoft Graph et télécharger les modèles Simulink sans risque d'interception ou d'usurpation.

d. Automatisation de la chaîne HIL : Flux de bout en bout

Le cœur métier du PING est la refonte complète du flux de configuration des bancs de tests HIL, avec pour objectif la suppression totale des saisies manuelles. L'architecture de ce traitement automatisé s'appuie sur une interopérabilité forte entre MATLAB et Python pour exploiter les forces de chaque langage.

Le flux de données automatisé se décompose en trois étapes séquentielles :

- Extraction des données sources (MATLAB/Simulink) :
Plutôt que d'analyser les modèles à la main, un script MATLAB exploitera l'API native de Simulink. Il parcourra l'arbre du modèle (.slx) pour en extraire l'architecture, les paramètres de configuration et les blocs pertinents. MATLAB est ici le choix technologique le plus robuste pour interagir avec son propre format propriétaire.
- Génération du fichier de configuration (Python) :
Une fois les données extraites par MATLAB et croisées avec les variables génériques issues de l'A2L (converties en YAML lors du S8), elles sont transmises à l'environnement Python. Un script Python prend alors le relais pour formater, structurer et générer automatiquement le fichier Excel de configuration pivot (le fichier PIV). Python est privilégié ici pour son extrême efficacité dans la manipulation de données tabulaires (via des bibliothèques telles que pandas ou openpyxl).
- Génération des scripts de tests finaux :
Enfin, le générateur développé au Semestre 8 ingère ce fichier PIV parfaitement formaté pour écrire les scripts de tests automatisés qui seront exécutés sur le banc HIL.

Cette chaîne de traitement (Simulink → Python → Excel PIV → Script de Test) transforme une procédure manuelle chronophage et faillible en un pipeline logiciel fluide, rapide et hautement fiable.

e. Implémentation technique de l'extraction automatisée (Backend)

Le découplage entre l'extraction des données graphiques de modélisation et l'écriture du fichier pivot Excel nécessite une communication inter-environnement rigoureuse. Afin d'éliminer la lecture visuelle humaine des fichiers .slx, le backend de la Toolbox tire parti des couches de programmation natives de MathWorks.

Pour inspecter le modèle de contrôle sans instancier l'interface lourde de Simulink (ce qui ralentirait le processus), le script MATLAB s'exécute en mode silencieux. Il

utilise l'API de commande pour charger le modèle en mémoire, lister l'intégralité des nœuds et extraire les métadonnées.

Voici l'architecture du script MATLAB développé pour la phase d'extraction des ports et paramètres :

```
function model_metadata = extract_simulink_nodes(model_name)
    % Chargement du modèle en mémoire sans ouverture de la fenêtre
    graphique
    load_system(model_name);

    % Extraction récursive de tous les blocs de type "Inport" (Entrées)
    raw_inports = find_system(model_name, 'SearchDepth', 3, 'BlockType',
    'Inport');

    % Initialisation de la structure de stockage des métadonnées
    model_metadata = struct();

    for i = 1:length(raw_inports)
        current_block = raw_inports{i};
        % Récupération des propriétés critiques du signal
        name = get_param(current_block, 'Name');
        data_type = get_param(current_block, 'OutDataTypeStr');
        dimensions = get_param(current_block, 'PortDimensions');

        % Structuration des données associées au bloc
        model_metadata.inputs(i).name = name;
        model_metadata.inputs(i).type = data_type;
        model_metadata.inputs(i).dim = dimensions;
    end

    % Libération de la mémoire et fermeture du modèle sans sauvegarde
    close_system(model_name, 0);
end
```

Les données collectées dans cette structure MATLAB sont immédiatement sérialisées dans un fichier temporaire JSON.

L'environnement Python prend alors le relais en arrière-plan. Il ingère ce JSON via la bibliothèque pandas pour générer un Dataframe structuré. C'est ce composant Python qui réalise les alignements finaux avec les variables du fichier YAML (généré à partir de l'A2L) et pilote le formatage final des feuilles de calcul du fichier Excel PIV à l'aide du moteur d'écriture openpyxl.

VII. CHAPITRE 4 : INGÉNIERIE LOGICIELLE, QUALITÉ ET PLANIFICATION

La réussite d'un projet d'intégration d'une telle envergure ne repose pas uniquement sur des choix d'architecture, mais également sur la rigueur des méthodes de développement et de validation. Ce chapitre détaille les processus d'ingénierie logicielle mis en place, la stratégie d'assurance qualité envisagée, ainsi que la planification macroscopique des jalons pour la suite du Projet Ingénieur.

a. Gestion de configuration et travail collaboratif

Pour assurer la traçabilité et la sécurité du code source de la vingtaine d'outils, le cycle de développement s'appuie sur le système de contrôle de version Git.

L'architecture des dépôts est structurée pour favoriser le travail collaboratif sans risquer de corrompre les outils en production :

- Environnements de développement isolés : Les développements de nouvelles fonctionnalités ou les corrections de bugs sont effectués sur des dépôts personnels ou des branches dédiées. Cela permet d'isoler les expérimentations (comme la création du moteur de rendu HTML) du code stable.
- Dépôt centralisé (xp_legacy) : Ce dépôt fait office de source de vérité unique (Single Source of Truth) pour toute l'équipe Outils, garantissant que chaque membre dispose de la dernière version du code métier.

Actuellement, l'organisation de l'équipe repose sur une forte autonomie : après validation fonctionnelle en local de mes développements, je dispose des droits nécessaires pour fusionner directement (Merge) mon code dans le dépôt principal, sans processus formel de revue de code (Code Review). Ce fonctionnement particulièrement agile témoigne de la confiance accordée par mon maître d'apprentissage. Cependant, dans une optique d'amélioration continue, l'introduction de Pull Requests (PR) formelles constituera une évolution logique de nos processus pour sécuriser davantage l'intégration à mesure que la Toolbox se complexifiera.

b. Stratégie de Branching (Gestion des branches Git)

Bien que je dispose des droits de fusion directs sur le dépôt principal, j'applique une méthodologie stricte de gestion de versions s'inspirant du modèle Feature Branch Workflow. Le dépôt xp_legacy est protégé contre les modifications directes sur sa branche principale (main ou master), qui doit toujours contenir un code stable et déployable en production.

Pour chaque nouvelle fonctionnalité (ex: création de l'interface JSON) ou correction de bug, le flux de travail est le suivant :

- Création d'une branche isolée (ex: feature/json-renderer ou hotfix/regex-a2l).

- Développements locaux et commits atomiques (un commit = une action logique).
- Poussée (Push) de la branche sur le dépôt distant.
- Fusion (Merge) sur la branche principale après validation fonctionnelle locale.

De plus, pour éviter de polluer le dépôt distant avec des fichiers temporaires, de configuration locale ou des binaires lourds, j'ai mis en place un fichier `.gitignore` strict adapté à notre environnement hybride. Il exclut notamment les fichiers compilés Python (`__pycache__`/, `*.pyc`), les logs d'exécution, et les fichiers d'environnement virtuel (`venv/`).

c. Usine logicielle : Stratégie CI/CD et Assurance Qualité

L'un des objectifs SMART du PING est de garantir la fiabilité absolue des outils fournis aux ingénieurs d'essais. Actuellement, les tests sont réalisés manuellement, ce qui est chronophage et source de régressions potentielles lors de la mise à jour d'un outil.

Pour pallier ce problème, le projet prévoit la mise en place d'un pipeline d'Intégration et de Déploiement Continu (CI/CD). La définition exacte de cette architecture constitue la prochaine étape critique du projet. N'ayant pas encore débuté la phase de recherche technique sur les tests automatisés, l'objectif prioritaire sera d'identifier les éléments à tester en premier (prioriser la validation mathématique des scripts Python ou le bon rendu des interfaces HTML) et de s'orienter vers les axes suivants :

- La définition des frameworks de tests : Évaluer et implémenter les solutions les plus adaptées pour notre environnement hybride (par exemple, le MATLAB Unit Test Framework pour le backend de routage, et pytest pour les algorithmes Python d'extraction de données).
- L'automatisation via le dépôt distant : Configurer le dépôt `xp_legacy` (via des outils tels que GitHub Actions ou GitLab CI) pour qu'à chaque fusion d'un nouveau code, une suite de tests automatisés s'exécute virtuellement.
- Le déploiement et la mise à jour continue : Concevoir un mécanisme permettant à la FEVio Toolbox de vérifier de manière transparente la disponibilité d'une nouvelle version validée et de proposer une mise à jour en arrière-plan à l'utilisateur.

d. Maquette de l'architecture CI/CD cible (GitHub Actions)

La mise en place de l'Intégration Continue (CI) nécessitera l'utilisation d'un orchestrateur. Dans le cas d'une utilisation de GitHub, le moteur GitHub Actions permettra d'automatiser les tests à chaque nouveau push sur le dépôt.

Le pipeline virtuel (Workflow) devra provisionner un environnement temporaire, installer les dépendances (Python) et configurer l'environnement MATLAB (matlab-actions) avant d'exécuter les suites de tests.

Bien qu'en phase d'étude, voici la maquette d'architecture du fichier de configuration YAML (.github/workflows/ci.yml) que je prévois d'implémenter pour tester automatiquement le module d'extraction A2L et le moteur de rendu :

```
name: FEVio Toolbox CI Pipeline

on:
  push:
    branches: [ "main", "feature/*" ]
  pull_request:
    branches: [ "main" ]

jobs:
  test-python-backend:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository code
        uses: actions/checkout@v3

      - name: Set up Python 3.14
        uses: actions/setup-python@v4
        with:
          python-version: '3.14'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install pytest pyyaml pandas openpyxl

      - name: Run Python Unit Tests (A2L Converter & PIV Generator)
        run: pytest tests/python_backend/

  test-matlab-core:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository code
        uses: actions/checkout@v3

      - name: Set up MATLAB
        uses: matlab-actions/setup-matlab@v2

      - name: Run MATLAB Unit Tests (Routing & Simulink API)
        uses: matlab-actions/run-command@v2
        with:
          command: results = runtests('tests/matlab_core');
          assertSuccess(results);
```

Cette automatisation garantira que toute modification du code métier Python ou du routeur MATLAB n'introduit aucune régression (cassure) dans les fonctionnalités existantes de la Toolbox.

e. Planification et Roadmap (S9 - S10)

Le déploiement de la FEVio Toolbox et la migration de l'écosystème logiciel s'étaleront sur les 12 prochains mois. La feuille de route s'articule autour des jalons suivants :

- Jalon 1 : Été 2026 (Stabilisation et Core Toolbox)
 - Finalisation et fiabilisation des scripts de tests Python générés depuis les fichiers PIV (sortie de la phase Bêta).
 - Lancement des développements pour le module d'extraction native depuis Simulink.
 - Phase de recherche sur les frameworks de tests automatisés (CI).
 - Livraison de la version V1 de l'interface globale de la Toolbox.
- Jalon 2 : Semestre 9 - Automne/Hiver 2026 (Sécurisation, CI/CD et Nouveaux Outils)
 - Déploiement du module de base de données sécurisée via Microsoft Entra ID (sous réserve de validation finale par la direction IT FEV).
 - Implémentation effective des pipelines CI/CD sur le dépôt xp_legacy.
 - Début du développement du nouvel outil de génération de rapports automatiques unifiés.
 - Début de la migration itérative des 20 outils existants vers le nouveau modèle hybride HTML/MATLAB.
- Jalon 3 : Semestre 10 - Printemps 2027 (Déploiement massif et Clôture)
 - Achèvement de la conversion et de l'intégration de la totalité des outils dans la Toolbox.
 - Finalisation et déploiement du générateur de rapports.
 - Campagne de tests finaux, optimisation des performances du moteur de rendu JSON.
 - Rédaction de la documentation globale (User Guides centralisés) et passation du projet.

VIII. CHAPITRE 5 : VALIDATION DES COMPÉTENCES INGÉNIEUR ESEO

Le Projet Ingénieur (PING), de sa phase préparatoire (S8) à son déploiement final (S10), constitue un cadre idéal pour la mise en pratique et la validation des sept compétences fondamentales du référentiel ESEO. La nature du projet, alliant architecture logicielle, cybersécurité et interaction avec les utilisateurs, permet de couvrir l'intégralité de ce spectre.

a. Analyser et Diagnostiquer

J'ai dû auditer un patrimoine logiciel complexe d'une vingtaine d'outils et diagnostiquer ses faiblesses profondes. Cela s'est traduit par l'identification des vulnérabilités liées à l'obsolescence de Python 2.7.2, l'analyse des limitations techniques de MATLAB App Designer, et la mise en évidence des failles de sécurité dans la gestion des modèles Simulink.

b. Concevoir

Face aux limites identifiées, j'ai dû concevoir des architectures innovantes. La conception de la FEVio Toolbox ne s'est pas limitée à une simple maquette graphique. J'ai architecturé une solution hybride complexe séparant le frontend (généré dynamiquement par un moteur JavaScript lisant un fichier JSON) du backend (MATLAB). Parallèlement, j'ai défini l'architecture réseau cible en adoptant une approche Zero Trust pour l'intégration de Microsoft Entra ID.

c. Produire et Réaliser

La concrétisation de ces architectures s'est manifestée par la production d'un code robuste et optimisé. Mes réalisations incluent le développement de la logique de parsing (via expressions régulières et bibliothèques ASAM) pour le convertisseur A2L vers YAML, ainsi que la refactorisation des scripts d'automatisation Intecrio en Python 3.14.3. J'ai veillé à appliquer des standards de codage modernes pour faciliter la maintenabilité.

d. Valider

La validation technique a été un fil conducteur du Semestre 8. Outre la viabilité de la communication bidirectionnelle entre JavaScript et MATLAB, j'ai mis en place un processus de validation sur le terrain pour les outils critiques. À titre d'exemple, la première version bêta du générateur de scripts (Simulink $\rightarrow$ PIV $\rightarrow$ Python) a été livrée à un stagiaire en charge des essais sur banc HIL afin d'éprouver le code généré en conditions réelles. La suite du projet renforcera cette compétence avec l'implémentation d'une usine logicielle (CI/CD) pour garantir la non-régression de l'ensemble du code.

e. Piloter (et s'organiser)

L'autonomie qui m'a été accordée m'a permis de développer mes compétences en pilotage. Je gère aujourd'hui la version de mes codes via Git, effectuant des fusions directes sur le dépôt central xp_legacy. Ce pilotage s'évalue également par la mesure du retour sur investissement (ROI) des solutions déployées. Par exemple, l'automatisation apportée sur la RP App a permis de réduire le temps de génération manuel de deux semaines à une simple demi-journée. De même, d'autres scripts plus simples permettent d'économiser plusieurs heures de travail hebdomadaire aux ingénieurs, prouvant la valeur ajoutée du projet.

f. S'adapter

Le contexte industriel impose une forte capacité d'adaptation, notamment pour répondre au plus près aux besoins et aux usages des utilisateurs finaux. Confronté aux contraintes de sécurité de l'entreprise et aux limites ergonomiques natives de MATLAB, j'ai su sortir de ma zone de confort. J'ai ainsi intégré des technologies de développement web (HTML, CSS, JavaScript) que je ne maîtrisais pas initialement, afin de concevoir une interface moderne et intuitive. Parallèlement, j'ai adapté l'architecture de mes algorithmes pour faire communiquer de manière fluide un écosystème logiciel hétérogène, garantissant ainsi que l'outil final s'intègre parfaitement dans les processus métiers quotidiens des ingénieurs.

g. Communiquer

En tant que développeur d'outils, la communication est primordiale pour assurer l'adoption des solutions. J'accompagne chaque livraison majeure par la rédaction de notes de version (Release Notes) détaillant les nouveautés et correctifs, lesquelles sont spécifiquement diffusées aux ingénieurs. Par ailleurs, je réalise régulièrement des démonstrations en direct (live demos) lors des réunions hebdomadaires de l'équipe pour présenter les nouvelles fonctionnalités et recueillir les retours à chaud. Enfin, la rédaction de la première version du manuel utilisateur (User Guide) de la Toolbox participe activement à la conduite du changement auprès de l'ensemble des destinataires de la livraison.

IX. CONCLUSION

Le Semestre 8 a marqué une étape décisive dans la structuration de mon Projet Ingénieur (PING) au sein de FEV France. Face à une problématique industrielle complexe, caractérisée par un écosystème logiciel fragmenté, des processus de tests chronophages et des vulnérabilités de cybersécurité naissantes, il était impératif de repenser l'approche de l'équipe « Outils ».

Les travaux préparatoires menés durant ces six derniers mois ont permis de valider la faisabilité technique des solutions de rupture envisagées. La migration critique des environnements vers Python 3.14.3 a assaini la dette technique, tandis que le développement du module de conversion A2L vers YAML a prouvé notre capacité à manipuler des données automobiles brutes sans intervention manuelle. Plus important encore, la conception d'une architecture d'interface hybride, exploitant un moteur de rendu JavaScript couplé à un backend MATLAB, a démontré qu'il était possible d'offrir une expérience utilisateur moderne, unifiée et modulaire, tout en contournant les contraintes de licences et d'obsolescence visuelle.

Ces preuves de concept techniques constituent un socle solide pour le déploiement du framework cible : la FEVio Toolbox. Si certains objectifs, comme l'intégration complète des pipelines de tests automatisés (CI/CD) ou la validation finale par la direction IT de l'architecture Zero Trust sous Microsoft Entra ID, restent à concrétiser, la feuille de route est désormais clairement tracée pour les semestres à venir.

Au-delà de l'aspect purement technologique, ce projet m'a permis de prendre la mesure des enjeux de l'ingénierie logicielle en milieu industriel. L'amélioration continue des processus, illustrée par la réduction drastique des temps de configuration (passant de deux semaines à une demi-journée sur certaines tâches), confirme la valeur ajoutée de notre démarche. Les phases de déploiement à venir (Semestres 9 et 10) exigeront une rigueur accrue en matière de qualité logicielle et d'accompagnement des utilisateurs, me permettant ainsi de consolider pleinement mes compétences de futur ingénieur.

X. RÉFÉRENCES BIBLIOGRAPHIQUES ET WEBOGRAPHIE

Documentation technique et architecture :

- MathWorks. Documentation officielle MATLAB : uihtml - Create HTML component in UI figure. Disponible sur : <https://fr.mathworks.com/help/matlab/ref/uihtml.html>
- Python Software Foundation. Python 3.14 Documentation. Disponible sur : <https://docs.python.org/3.14/>
- Microsoft. Microsoft identity platform and OAuth 2.0 authorization code flow with PKCE. Documentation Microsoft Learn pour Entra ID. Disponible sur : <https://learn.microsoft.com/>
- ASAM e.V. ASAM MCD-2 MC (A2L) - Data model for ECU measurement and calibration. Spécifications officielles du standard.

Documentation interne (FEV France) :

- FEV Group. Directives internes de Cybersécurité et politique de gestion des terminaux (EDR). (Document interne et confidentiel).
- Équipe Outils FEV. Code source hérité (Legacy) et spécifications de la RP App. (Dépôts Git internes).

XI. GLOSSAIRE

A2L (ASAM MCD-2 MC) : Format de fichier standardisé dans l'industrie automobile décrivant la topologie mémoire, les variables de mesure et les paramètres de calibration d'un calculateur électronique.

BMS (Battery Management System) : Système électronique permettant la gestion et le contrôle des batteries d'accumulateurs.

CI/CD (Continuous Integration / Continuous Deployment) : Pratique d'ingénierie logicielle consistant à automatiser les tests et le déploiement du code à chaque modification.

ECU (Electronic Control Unit) : Calculateur électronique embarqué dans un véhicule gérant une ou plusieurs fonctions (moteur, freinage, batterie, etc.).

EDR (Endpoint Detection and Response) : Solution de cybersécurité agissant sur les postes de travail pour détecter et bloquer les comportements malveillants ou les exécutables non reconnus.

Entra ID (anciennement Azure AD) : Service de gestion des identités et des accès basé sur le cloud de Microsoft, utilisé pour l'authentification sécurisée des collaborateurs FEV.

HIL (Hardware-in-the-Loop) : Technique de simulation permettant de tester un calculateur réel (Hardware) en le connectant à un environnement virtuel simulant le reste du véhicule (la boucle / Loop).

JSON (JavaScript Object Notation) : Format de données textuel léger et lisible par l'humain, utilisé pour représenter des données structurées et communiquer entre le frontend (JavaScript) et le backend (MATLAB).

JWT (JSON Web Token) : Standard cryptographique permettant d'échanger des informations d'authentification de manière sécurisée et vérifiable.

MFA (Multi-Factor Authentication) : Processus d'authentification exigeant plusieurs preuves d'identité (ex: mot de passe + validation sur smartphone).

PIV (Fichier Pivot) : Dans le contexte de FEV, fichier Excel de configuration formaté servant d'intermédiaire standardisé entre les modèles Simulink et les générateurs de scripts Python.

PoC (Proof of Concept) : Preuve de concept. Réalisation expérimentale visant à démontrer la faisabilité technique d'une idée ou d'une architecture.

SSO (Single Sign-On) : Authentification unique permettant à un utilisateur d'accéder à plusieurs applications avec un seul compte d'entreprise.

YAML (YAML Ain't Markup Langage ou Yet Another Markup Language) : Format de sérialisation de données lisible par l'homme, couramment utilisé pour les fichiers de configuration.

ETAS : Société spécialisée dans le développement d'outils, de logiciels et de solutions matérielles intégrées pour les systèmes embarqués de l'industrie automobile, notamment pour la mesure, la calibration et le prototypage rapide.

Intecrio (ETAS Intecrio) : Suite logicielle d'ingénierie développée par ETAS, dédiée au prototypage rapide de fonctions de contrôle-commande. Elle permet d'intégrer, de configurer et de tester des modèles hétérogènes (Simulink, code C) sur des cibles matérielles ou de simulation avant leur implémentation finale sur calculateur.

RP App (Rapid Prototyping Application) : Boîte à outils (Toolbox) logicielle développée en interne et destinée aux équipes de prototypage rapide pour automatiser les flux de configuration des modèles de contrôle. Ce module spécifique a fait l'objet d'une refonte critique pour migrer son architecture de Python 2.7.2 vers Python 3.14.3.